

1. Descripción de las tareas a realizar

Proyecto : Sistema de Gestión de Pedidos y E-commerce

1. Contexto

El objetivo de este proyecto es desarrollar una aplicación web robusta para la gestión de una tienda online. La aplicación debe ser capaz de gestionar el ciclo de vida completo de un pedido, desde la navegación del catálogo hasta la confirmación de pago, aplicando una arquitectura profesional y garantizando la integridad de los datos.

Funcionalidades:

- Catálogo Dinámico: Carga y visualización de categorías y productos desde la base de datos.
- Gestión de Ventas: Implementación de un flujo de carrito de compra y pasarela de pago.
- Control de Pedidos: Sistema para monitorizar y actualizar los estados de los pedidos (Pendiente, Pagado, Enviado).
- Panel Administrativo (CRUD): Gestión completa de productos y pedidos para usuarios con privilegios.
- Sistema de Usuarios: Registro, login y control de sesiones.
- Notificaciones: Envío de correos electrónicos de confirmación mediante librerías externas.

2. Requisitos Técnicos y Objetivos de Aprendizaje

El proyecto será evaluado bajo los siguientes criterios de calidad y buenas prácticas de programación:

A. Arquitectura y Patrones de Diseño

- Patrón MVC : Separación estricta de responsabilidades. La lógica de negocio, la presentación y la gestión de datos deben estar desacopladas.
- Patrón Repository: Implementación de una capa de abstracción para el acceso a datos, garantizando un código limpio, reutilizable y fácil de testear.
- El acceso a la aplicación se realizará exclusivamente a través de `public/index.php`, manteniendo el resto de la estructura fuera del documento raíz del servidor web.
- Uso de una clase Router para gestionar rutas amigables y dirigir las peticiones a los controladores correspondientes.

B. Gestión de Datos y Persistencia

- Acceso a Datos con PDO: Uso obligatorio de la interfaz PDO con enfoque orientado a objetos para todas las interacciones con la base de datos.

- Uso sistemático de sentencias preparadas para prevenir ataques de Inyección SQL.
- Validación Robusta: Implementación de validaciones en el lado del servidor para todos los formularios, proporcionando feedback claro al usuario mediante mensajes de error.

C. Herramientas y Gestión de Dependencias

- Composer: Uso de autoloader para la carga automática de clases (estándar PSR-4).
- Librerías Externas: Integración de paquetes vía Composer para tareas específicas:
 - Gestión de variables de entorno (.env) para ocultar credenciales sensibles.
 - Generación de documentos PDF para facturas.
 - Sistemas de paginación y generación de tokens de seguridad (CSRF).
 - Envío de correos electrónicos (ej. PHPMailer).

D. Seguridad y Sesiones

- Uso adecuado de sesiones y cookies para la gestión del carrito y la autenticación, siguiendo protocolos de seguridad para evitar secuestro de sesiones.
- Programación de un sistema de registro y login seguro (hasheo de contraseñas, control de accesos por roles).

3. Estándares de Calidad

Para considerar la práctica como apta, el código debe cumplir con:

1. Legibilidad: Código organizado y legible siguiendo estándares.
2. Aplicación del principio DRY (*Don't Repeat Yourself*), evitando duplicidad mediante el uso de clases y funciones eficientes.
3. Documentación: Comentarios pertinentes que expliquen la lógica compleja del negocio.
4. Funcionalidad: Salidas válidas que correspondan estrictamente a los requisitos del guion.

1.1. Requisitos detallados

Se recuerda que el sistema debe gestionar el flujo completo de venta, desde la administración del catálogo hasta la formalización de pedidos, garantizando la persistencia de datos y la seguridad en el acceso según el rol del usuario.

Requisitos

A. Gestión de Catálogo y Usuarios

- Autenticación y Autorización: El sistema requiere identificación obligatoria para cualquier operación. Se gestionarán dos roles: Cliente y Administrador.
- Módulo de Categorías:
 - Consulta pública
 - Alta de nuevas categorías. (sólo usuarios administradores)
- Módulo de Productos:
 - Consulta detallada del catálogo.
 - Sólo para administradores: CRUD completo (Crear, Leer, Actualizar y Borrar).
- Carrito:
 - Adición de múltiples unidades de productos al carrito.
 - Gestión de línea de pedido: visualización y eliminación de productos antes de confirmar.

B. Ciclo de Vida del Pedido

- Formalización: Al finalizar la compra, el sistema debe persistir el pedido en la base de datos y enviar un correo electrónico automático de confirmación.
- Gestión de Pagos: Implementación de un módulo para la tramitación del pago.
- Control de Estados: Los pedidos se registrarán inicialmente con el estado "Pendiente" . Los administradores podrán consultar y supervisar estos registros.

Para simplificar, los datos de envío se solicitan en el momento del pedido. Se recomienda diseñar el sistema de forma extensible para permitir, en futuras versiones, la gestión de múltiples direcciones y perfiles de facturación.

Modelo de Datos y Entidades

Se debe diseñar una base de datos normalizada que contemple, al menos, las siguientes entidades y atributos:

Entidad	Atributos Clave	Observaciones
Usuario	Código, Nombre, Apellidos, Email (login), Clave, Rol.	El email actúa como identificador único.
Categoría	Código, Nombre, Descripción.	Relación 1:N con Productos.

Producto	Código, Nombre, Descripción, Precio, Stock, Oferta, Fecha, Imagen.	Cada producto pertenece a una única categoría.
Pedido	Código, Dirección, Provincia, Localidad, Coste total, Estado, Fecha, Hora.	Se asocia a un usuario.
Línea Pedido	ID, Unidades, Código Producto, Código Pedido.	Detalle granular de cada transacción.

Es imperativo que antes de comenzar la codificación, se realice un Análisis de Requisitos exhaustivo. Este análisis debe incluir:

1. Diagrama Entidad-Relación de la base de datos.
2. Mapa de rutas (endpoints) de la aplicación.
3. Definición de las responsabilidades de cada Clase/Controlador.

2. Diseña y crea la base de datos

Una de las cosas que tendremos claras después del análisis de requisitos será que datos van a ser persistentes, así como el repositorio donde se van a almacenar.

En este primer ejemplo usaremos una base de datos relacional y para su diseño se usará el esquema Entidad-Relación.

A continuación, se muestra un diagrama con un ejemplo básico de lo que puede ser la base de datos de esta aplicación.

Y un ejemplo con el script en el que puedes hacer los cambios que necesites para tu aplicación.

Si haces cambios deberás aportar el archivo que permita la creación completa de la base de datos usada y un diagrama o esquema Entidad-Relación de la misma.

2.1. Entidad Relación

Sugerencia de Modelo Entidad Relación



BASE DE DATOS:

```
-- =====  
-- Base de datos: tienda  
-- =====
```

```
CREATE DATABASE IF NOT EXISTS tienda  
  CHARACTER SET utf8mb4  
  COLLATE utf8mb4_unicode_ci;
```

```
USE tienda;
```

```
-- -----  
-- USUARIOS  
-- -----
```

```

CREATE TABLE `usuarios` (
  `id`      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nombre`  VARCHAR(60)    NOT NULL,
  `apellidos` VARCHAR(60)  NOT NULL,
  `email`   VARCHAR(255)   NOT NULL,
  `password` VARCHAR(255)  NOT NULL, -- hash
  `rol`     ENUM('admin','usuario') NOT NULL DEFAULT 'usuario',
  `confirmado` BOOLEAN     NOT NULL DEFAULT FALSE,
  `token`   VARCHAR(255)   DEFAULT NULL,
  `token_exp` DATETIME     DEFAULT NULL,
  `created_at` TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uq_usuarios_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-----
-- Categorías
-----

```

```

CREATE TABLE IF NOT EXISTS categorias (
  id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nombre  VARCHAR(100) NOT NULL,
  descripcion TEXT,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_categorias PRIMARY KEY (id),
  CONSTRAINT uq_cat_nombre UNIQUE (nombre)
) ENGINE=InnoDB;

```

```

-----
-- Productos
-----

```

```

CREATE TABLE IF NOT EXISTS productos (
  id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  categoria_id INT UNSIGNED NOT NULL,
  nombre  VARCHAR(150) NOT NULL,
  descripcion TEXT,
  precio  DECIMAL(10,2) NOT NULL CHECK (precio >= 0),
  precio_oferta DECIMAL(10,2) DEFAULT NULL,
  stock   INT UNSIGNED NOT NULL DEFAULT 0,
  activo  TINYINT(1)   NOT NULL DEFAULT 1,
  imagen  VARCHAR(255),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_productos PRIMARY KEY (id),
  CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id)

```

```
REFERENCES categorias (id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- Índice para búsquedas por categoría y filtrado de activos
CREATE INDEX idx_productos_categoria ON productos (categoria_id, activo);
```

```
-- Pedidos
```

```
CREATE TABLE IF NOT EXISTS pedidos (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  usuario_id  INT UNSIGNED NOT NULL,
  provincia   VARCHAR(100) NOT NULL,
  localidad   VARCHAR(100) NOT NULL,
  direccion   VARCHAR(255) NOT NULL,
  subtotal    DECIMAL(12,2) NOT NULL DEFAULT 0.00 CHECK (subtotal >= 0),
  impuestos   DECIMAL(12,2) NOT NULL DEFAULT 0.00 CHECK (impuestos >= 0),
  coste_total DECIMAL(12,2) NOT NULL DEFAULT 0.00 CHECK (coste_total >= 0),
  estado      ENUM(
    'pendiente',
    'confirmado',
    'pagado',
    'enviado',
    'entregado',
    'cancelado'
  ) NOT NULL DEFAULT 'pendiente',
  fecha_pedido DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_pedidos PRIMARY KEY (id),
  CONSTRAINT fk_pedido_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuarios (id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- Índice para consultar pedidos por usuario y estado
CREATE INDEX idx_pedidos_usuario ON pedidos (usuario_id, estado);
```

```
-- Líneas de pedido
```

```
CREATE TABLE IF NOT EXISTS lineas_pedidos (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  pedido_id   INT UNSIGNED NOT NULL,
  producto_id INT UNSIGNED NOT NULL,
  unidades    SMALLINT UNSIGNED NOT NULL CHECK (unidades > 0),
  precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario >= 0),
  subtotal_linea DECIMAL(12,2) NOT NULL
    GENERATED ALWAYS AS (unidades * precio_unitario) STORED,
  CONSTRAINT pk_lineas_pedidos PRIMARY KEY (id),
  CONSTRAINT fk_linea_pedido FOREIGN KEY (pedido_id)
```

```
REFERENCES pedidos (id) ON DELETE CASCADE ON UPDATE CASCADE,  
CONSTRAINT fk_linea_producto FOREIGN KEY (producto_id)  
REFERENCES productos (id) ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Diseña la estructura básica de tu web. No repliques el ejemplo que se usará en clase.

Intenta realizar el diagrama de flujo de pantallas por las que pasa el usuario al realizar una operación.

El siguiente ejemplo no tiene por qué coincidir con el diseño de tu aplicación:

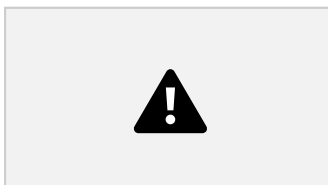


Recordemos que la idea básica de este patrón es separar nuestros sistemas en tres capas, el Modelo, la Vista y el Controlador.

- El Modelo se encarga de todo lo que tiene que ver con la persistencia de datos. Guarda y recupera la información del medio persistente que utilizemos, ya sea una base de datos, ficheros de texto, XML, etc.
- La Vista presenta la información obtenida con el modelo de manera que el usuario la pueda visualizar.
- El Controlador, dependiendo de la acción solicitada por el usuario, es el que pide al modelo la información necesaria e invoca a la plantilla(de la vista) que corresponda para que la información sea presentada.

Por lo tanto, vamos a crear una estructura de directorios que nos permita implementar el Modelo Vista Controlador.

Comenzamos creando una estructura de directorios parecida a la siguiente:



Añade una carpeta para guardar el script con la creación de la base de datos y sus tablas. De esta forma siempre estará disponible para la corrección de este ejemplo.



También es aconsejable que crees una carpeta para tener tus hojas de estilo y las imágenes que usarás en el diseño de tu aplicación.

En cada una de estas carpetas podemos crear también los archivos que ya sabemos que serán necesarios.

Por ejemplo, en la carpeta `models` tendremos que tener un archivo para cada una de las clases que nos permitirán recuperar la información de la base de datos. Por tanto, crea los archivos `usuario.php`, `pedido.php`, `categoria.php` y `producto.php`.



En la carpeta `views` crearemos una subcarpeta para cada una de las entidades de nuestra base de datos. Además, si hemos decidido dividir el código de nuestras páginas, para reutilizar las diferentes secciones como la cabecera o el pie, también incluiremos estos archivos en las vistas. Crea una carpeta específica para ello.



En la carpeta `controllers` crea los archivos `UsuarioController.php`, `PedidoController.php`, `CategoriaController.php` y `ProductoController.php`



Como puedes observar, he añadido dos más. Una para la gestión del carrito y otra para controlar los errores 404.

Cuando un recurso que se ha solicitado al servidor no está lo gestionamos con `ErrorController`.

Recuerda que esta estructura de carpetas se hace para que sea más didáctico el trabajo en clase. En un tu proyecto real los nombres de las carpetas no coincidirán.

5. Conexión a la base de datos

Asegúrate de que puedes conectarte a la base de datos.

Recuerda que puedes usar la clase BaseDatos que hemos creado en la unidad anterior.

6. Namespaces

Desde el principio haz uso de Namespaces.

No olvides declarar en cada una de las clases el namespace . Es decir la carpeta en la que está esa clase.

Será fundamental para que el autoloader de vendor, que instalamos vía Composer , pueda encontrar todas las clases que definirás en tu proyecto.

7. Rutas amigables

Para implementar un sistema de Rutas Amigables (Friendly URLs) en tu aplicación necesitamos transformar las URLs dinámicas tradicionales (incómodas y poco estéticas) en direcciones semánticas, limpias y optimizadas para el SEO.

En nuestra aplicación, las URLs serán legibles por humanos y reflejarán directamente la estructura del contenido. Una URL "amiga" o limpia se caracteriza por no mostrar parámetros técnicos como ? o =, ni exponer directamente el nombre del archivo .php (ej. index.php).

Para que el servidor Apache pueda procesar estas rutas, debemos asegurarnos de que el módulo mod_rewrite esté activo.

- En XAMPP: busca "Apache Modules" y asegúrate de que rewrite_module esté marcado.
- En configuración manual (httpd.conf): Descomenta la línea:

```
LoadModule rewrite_module modules/mod_rewrite.so
```

- Asegúrate de que la directiva AllowOverride All esté activa para el directorio de tu proyecto en la configuración de Apache, permitiendo que el archivo .htaccess tome el control.

Creación de los archivos .htaccess

Debes crear los tres archivos llamados `.htaccess` en la raíz de tu proyecto, en la carpeta `public` y en la carpeta `src`.

Gracias a estas reglas, el servidor realizará una traducción interna invisible para el usuario.

8. Registro de usuarios

El proceso de registro de nuevos clientes se diseñará siguiendo una arquitectura de responsabilidad única, utilizando controladores, servicios, repositorios y validadores dedicados para garantizar la seguridad y escalabilidad del sistema.

1. Validación y Sanitización: Clase `UserRequest`

Antes de que cualquier dato llegue a la lógica de negocio, la petición pasa por una clase especializada dentro de la carpeta `Request`. Esta clase tiene dos funciones críticas:

- Sanitización: Limpia los datos de entrada (eliminando espacios en blanco, etiquetas HTML peligrosas, etc.) para prevenir ataques XSS.
- Validación: Aplica reglas estrictas (ej. el email debe tener formato válido, la contraseña una longitud mínima). Si la validación falla, se interrumpe el flujo y se devuelven los errores a la vista.

2. El Controlador: `AuthController`

- Su método `register()` renderiza la vista `formregistro.php`.
- Su método `save()` recibe los datos ya procesados por la clase `Request`. En lugar de procesar la lógica, llama al `UsuarioService` para ejecutar la acción de registro.

3. Lógica de Negocio: `UsuarioService`

El `UsuarioService` recibe los datos limpios y se encarga de:

- Coordinar procesos adicionales (como verificar si el usuario ya existe).
- Llamar al `UsuarioRepository` para persistir la información.
- Podría, en un futuro, realizar otras acciones como enviar un email de bienvenida.
- No olvides que la contraseña siempre irá encriptada.

4. Capa de Datos: `UsuarioRepository`

Es la única clase con permiso para hablar con la base de datos. Recibe la información del Servicio y ejecuta la sentencia `INSERT` mediante sentencias preparadas, garantizando que el almacenamiento sea atómico y seguro.

5. Flujo de Mensajes y Sesiones

Finalmente, el flujo regresa a la vista. Utilizaremos Sesiones de PHP para comunicar el resultado al usuario:

- Éxito: Se genera `$_SESSION['register'] = "success"`.
- Error: Se genera `$_SESSION['errors']` con el listado de fallos.
- Para evitar que los mensajes persistan tras recargar la página, implementaremos una lógica de "limpieza de sesión" que elimine estas variables inmediatamente después de mostrarlas en el formulario.

8. Registro de usuarios

8.1. Modificar el registro de usuarios

Modifica el código para que los usuarios con el rol de administrador puedan crear usuarios con rol administrador o con rol 'user'.

El resto de usuarios sólo podrán hacer su propio registro.

Recuerda que una vez que se guarda un usuario en la base de datos ya puedes conocer otras propiedades como por ejemplo el id o el rol.

9. Identificación de usuarios(Login)

El proceso de autenticación permite verificar la identidad del usuario para concederle acceso a las funciones personalizadas de la tienda. Al igual que el registro, este flujo se centraliza en AuthController.

1. Captura y Validación: LoginRequest

El proceso comienza en la vista de login, que envía el email y la contraseña mediante POST. Antes de procesar la identificación:



La clase LoginRequest :

- intercepta los datos.
- Valida que el formato del email sea correcto y que ambos campos estén presentes.
- Realiza la sanitización para prevenir inyecciones de código.

El controlador recibe la petición validada y delega la comprobación al UsuarioService:

- El servicio solicita al UsuarioRepository que busque un registro que coincida con el email proporcionado (ya que el email es un campo único en nuestra base de datos).
- Si el usuario existe, el servicio utiliza la función de seguridad `password_verify($password, $usuario->password)`. Esta función comparará la contraseña en texto plano del formulario con el hash cifrado de la base de datos.
- Resultado:
 - Si coinciden, el servicio devuelve el objeto usuario completo.
 - Si fallan (email inexistente o contraseña errónea), el servicio lanza una excepción o devuelve false.

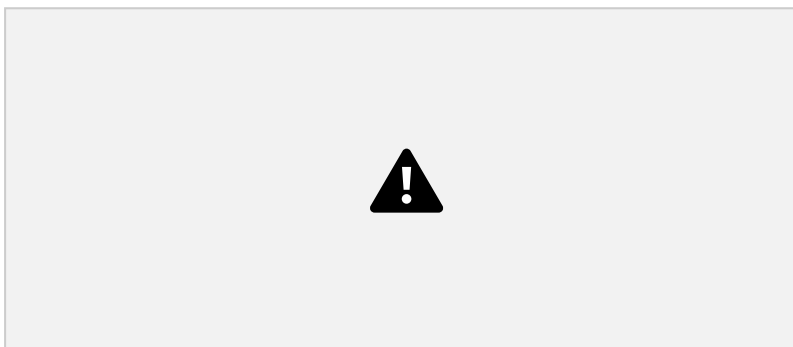
Una vez que el UsuarioService confirma que el usuario es válido:

1. El controlador guarda el objeto del usuario (o sus datos clave como ID, nombre, apellidos y rol) en una variable de sesión: `$_SESSION['identity'] = $usuario;`.

2. Se redirige al usuario a la página principal (index.php).

Para mejorar la experiencia de usuario y la navegación, realizaremos los siguientes cambios en las vistas:

- Página Principal: Mediante la sesión, daremos la bienvenida al usuario mostrando su nombre y apellidos de forma dinámica.
- Menú Condicional: Implementaremos una lógica en el encabezado (Header) que detecte si existe `$_SESSION['identity']`:
 - Si no está logueado: Muestra los enlaces de "Identificarse" y "Registrarse".
 - Si está logueado: Oculta los anteriores y muestra el nombre del usuario y el enlace de "Cerrar sesión".



10. Cerrar sesión

Tendremos que borrar la sesiones añadiendo al controlador UsuarioController el método `logout()`.

Este método borrará la sesión que tiene identificado al usuario.

11. Gestionar categorías

1. Funciones a implementar

Implementaremos las funciones básicas de creación, edición, borrado y listado de categorías.

Utilizaremos los siguientes componentes:

- CategoriaController: Es el responsable de dirigir las solicitudes del usuario hacia la lógica correspondiente y devolver la respuesta visual adecuada en cada caso.
- CategoriaService: Contiene la lógica de negocio (por ejemplo, asegurar que no existan nombres duplicados antes de crear).
- CategoriaRepository: Gestiona las consultas SQL (SELECT, INSERT) mediante sentencias preparadas para garantizar la seguridad.

- Categoría (Model): Representa la entidad categoría en el sistema.

Y no olvides que hay que validar y sanear los datos.

- CategoríaRequest: como mínimo el nombre de la categoría (comprobando que no esté vacío y que cumpla con una longitud mínima).

2. Protegiendo las rutas

La gestión de categorías está restringida exclusivamente a usuarios con el rol de administrador.

Por ello, debería de usarse un Middleware de autenticación que actúe como un filtro que intercepta peticiones antes de que lleguen al controlador.

El Middleware verifica la existencia de una sesión activa y valida y que el usuario tenga privilegios de administrador. Si el usuario no cumple los requisitos, el Middleware corta el flujo de ejecución y lo redirige automáticamente al login, a la página principal, o muestra un error de acceso denegado.

2.1. Creando el Middleware:

Crearemos la clase AdminMiddleware dedicada a interceptar la petición:

Su código puede ser algo parecido a :

```
class AdminMiddleware {
    public function handle() {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if (!isset($_SESSION['identity']) || $_SESSION['identity']->rol !== 'admin') {
            header("Location: " . base_url . "usuario/login");
            exit();
        }
    }
}
```

Y podríamos usarla desde el CategoriaController:

```

class CategoriaController {
    public function __construct() {
        // Ejecutamos el middleware al instanciar el controlador
        (new AdminMiddleware())->handle();
    }

    public function crear() {
        // Aquí ya sabemos con total certeza que el usuario es ADMIN
    }
}

```

Aunque si lo prefieres puedes mejorarlo devolviendo una respuesta HTTP lo que dará más flexibilidad .

```

interface MiddlewareInterface {
    public function handle(callable $next);
}

class AdminMiddleware implements MiddlewareInterface {
    public function handle(callable $next) {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if (!isset($_SESSION['identity']) || $_SESSION['identity']->rol !== 'admin') {
            http_response_code(403);
            echo "Acceso denegado. Solo administradores.";
            exit();
        }
        // Ejecuta la siguiente capa (controlador)
        return $next();
    }
}

```

Y la usaríamos desde el controlador:

```

class CategoríasController {
    private $repository;
    public function __construct() {
        $this->repository = new CategoriaRepository();
        // Ejecutar el middleware
        $middleware = new AdminMiddleware();
        $middleware->handle(fn() => true); // continúa normalmente
    }
    public function crear() {
        // Lógica de creación
    }
    public function editar($id) {
        // Lógica de edición
    }
    public function eliminar($id) {
        // Lógica de eliminación
    }
}

```

3. Vistas (Carpeta Views/categoría)

Diseñaremos, al menos, dos interfaces principales:

1. index.php (Listado): Una tabla o lista que muestra todas las categorías existentes obtenidas desde la base de datos.
2. crear.php (Formulario): Un formulario sencillo para capturar, al menos, el nombre de la nueva categoría.

12. Gestionar productos

En este módulo implementaremos la administración del inventario. Dado que un producto requiere estar asociado a una categoría, este apartado integrará elementos de ambos modelos.

1. Visualización y Control: ProductoController

La administración de productos se centralizará en el controlador ProductoController.

- Al igual que con las categorías, este controlador estará protegido por el AdminMiddleware, garantizando que solo los administradores puedan acceder a la gestión y creación.
- Método gestion(): Este método se encarga de solicitar al ProductoService el listado completo de artículos. Posteriormente, envía los datos a la vista views/productos/gestion.php para mostrarlos en una tabla.
 - *Nota:* Para pruebas iniciales, puedes insertar registros manualmente en la tabla productos vía SQL o phpMyAdmin.

2. Creación de Productos

El proceso de alta de un nuevo producto consta de dos fases:

A. El Formulario de Alta

El método crear() del controlador cargará la vista del formulario. Para que el usuario pueda asignar una categoría al producto, el controlador debe solicitar primero todas las categorías existentes al CategoriaService y pasarlas a la vista para generar un desplegable (<select>).

B. Procesamiento y Persistencia

Una vez enviado el formulario se procede a:

- La Validación y Sanitización (ProductoRequest): Se verifican precios, stocks y campos obligatorios.
- Gestión de Imágenes: No guardamos el archivo físico en la base de datos, solo su nombre. Se debe verificar si existe la carpeta de destino (ej. uploads/images); si no existe, debe crearla automáticamente. Posteriormente, se procesa la subida del

archivo. Intenta implementar una validación en el ProductService que renombre la imagen antes de guardarla (por ejemplo, usando un time() o un ID único) para evitar que dos productos con imágenes llamadas "foto.jpg" se sobrescriban.

3. Feedback al Usuario

Utilizaremos Sesiones para informar sobre el resultado de la operación:

- Si el producto se guarda con éxito, se genera `$_SESSION['product_save'] = "complete"`.
- En la vista de gestión se mostrará el mensaje de éxito y, acto seguido, se eliminará la sesión (unset) para que el aviso no reaparezca tras refrescar la página.

13. Mostrando productos

- Crea una vista que nos permita mostrar todos los productos de una categoría que haya sido seleccionada.
- Cuando la base de datos tenga un número elevado de productos, nuestra página debe de permitir cargarlos poco a poco. Puedes utilizar Composer para instalar una librería de paginación y con ella mostrar todos los productos de una categoría.

14. El carrito de la compra

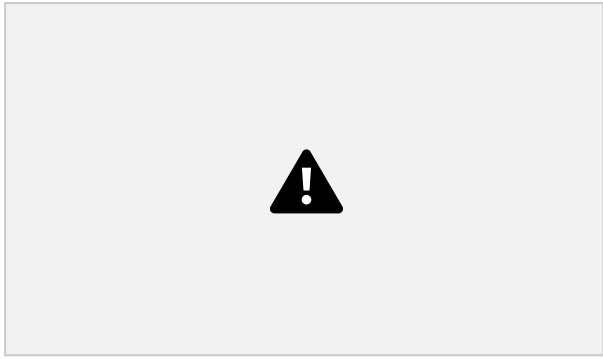
El carrito de la compra es el componente central de la experiencia de usuario. Su objetivo es permitir que cualquier visitante gestione una selección temporal de productos antes de proceder al pago.

1. Funcionamiento

- Acceso Universal: El carrito es accesible para todos los usuarios (identificados o no). La autenticación solo será obligatoria en el paso final de Confirmar Pedido.
- Persistencia Temporal: Utilizaremos la sesión del servidor (`$_SESSION['carrito']`) para mantener los productos mientras el usuario navega.
- Estructura de Datos: El carrito se almacenará como un array asociativo donde:
 - La Key es el `producto_id`.
 - El Value es la cantidad de unidades.

El array comenzará estando vacío y cuando se añada un producto al pedido, hay que comprobar si ya hay en el array algún elemento que tenga como clave el código del producto. Si no lo hay, se añade un nuevo elemento.

Por ejemplo, si el carrito está vacío y se añaden 2 unidades del producto con código 4, se creará un elemento del array con clave 4 y valor 2.



Ejemplo: Si posteriormente se añaden 3 unidades del producto con código 5, el carrito será:



En el caso de que haya un elemento con ese código, se suman las unidades al valor actual del elemento.

Ejemplo: Si se añaden otras cinco unidades del producto con código 4, el resultado será:



2. Lógica de Gestión: CarritoController

Este controlador centralizará todas las operaciones de modificación del array de sesión:

- Añadir Producto: Antes de insertar, el controlador verifica si el ID ya existe en el array.
 - Si no existe: Se crea la entrada con la cantidad inicial.
 - Si existe: Se incrementa el valor actual con las nuevas unidades.
- Modificar Unidades: Métodos específicos para incrementar (+1) o decrementar (-1) unidades de un producto existente.
- Eliminar Producto: Si las unidades llegan a cero o el usuario decide quitar el artículo, se elimina la llave del array mediante unset().
- Vaciar Carrito: Una acción que elimina por completo la variable `$_SESSION['carrito']`.

●

3. Visualización y Cálculos: La Vista del Carrito

Crearemos una vista dedicada (views/carrito/index.php) que transforme los IDs de la sesión en información legible:

- Dado que la sesión solo guarda IDs y cantidades, el controlador solicitará al ProductService los detalles (nombre, precio, imagen) de esos productos específicos para mostrarlos en una tabla.

No olvides que debemos de preparar el carrito para la vista y que hay que calcular el Total de Artículos y el Importe Total (Suma de precio * cantidad). Se recomienda añadir una columna de Subtotal por cada línea de producto para mejorar la claridad.

4. Flujo de Confirmación

La vista del carrito incluirá un botón de "Finalizar Pedido".

- Si el usuario está identificado (\$_SESSION['identity']), se le redirigirá directamente al formulario de dirección de envío.
- Si el usuario es un invitado, se le redirigirá al AuthController para que inicie sesión o se registre antes de procesar su compra.

Nota :es vital validar el stock real en la base de datos cada vez que un usuario añade un producto o incrementa unidades, evitando que se añadan al carrito artículos que ya no están disponibles.

15. Procesamiento del pedido

El proceso consiste en :

- insertar el pedido en nuestra base de datos
- Si el pedido se inserta correctamente:
 - decrementar el stock de los productos en la base de datos
 - mandar un correo de confirmación al cliente
 - vaciar el carrito

El método que se encarga de insertar el pedido en la base de datos recibe el carrito de la compra y el identificador del cliente que realiza el pedido.

Con esta información transformamos el carrito de la compra en un pedido, para lo cual:

- creará una nueva fila en la tabla pedidos y
- creará una fila en la tabla lineas_pedido por cada producto diferente que se pida. Ten en cuenta que necesitas usar la clave del nuevo pedido.

Las inserciones deben ocurrir como una transacción. Si alguna falla, hay que deshacer las anteriores (uso de commit).

Es muy importante que se actualice adecuadamente el stock de productos. Si el pedido se realiza tendremos que decrementar el stock de todos los productos que acabamos de vender. En caso contrario, no hay que modificarlo.

Si habéis observado los campos de la tabla "pedidos" habréis podido comprobar que existe el campo "estado". Los estados pueden ser, por ejemplo: confirmado, enviado, finalizado, etc.

Cuando procesemos un pedido debemos de informar al usuario por dos medios: en la propia web y enviando un correo con todos los datos del pedido: número de pedido, productos, cantidad de cada uno de ellos, total del importe, lugar de envío, etc.

16. Envío de correo

Una vez confirmado el pedido y finalizada la transacción en la base de datos, el sistema debe notificar al cliente de forma automática. Esta confirmación no solo mejora la experiencia de usuario, sino que sirve como comprobante legal de la compra.

1. Preparación del Entorno: PHPMailer y Composer

Utilizaremos PHPMailer y para instalarlo, utilizaremos el gestor de dependencias Composer: `composer require phpmailer/phpmailer`

2. La clase para gestionar el correo

Organizaremos la lógica de envío en una nueva clase que encapsulará el uso de PHPMailer para mantener el controlador limpio.

- Método de envío: Crearemos un método que reciba como parámetros el objeto del pedido, el carrito de la compra hidratado y el email del cliente.
- Contenido del correo: El cuerpo del mensaje se generará dinámicamente incluyendo:
 - Número de pedido y fecha.
 - Nombre personalizado del cliente.
 - Tabla detallada con productos, cantidades y subtotales.
 - Importe total y dirección de envío.

3. Seguridad y Configuración: El archivo .env

Nunca debemos escribir las contraseñas o credenciales SMTP directamente en el código. Esto se considera un fallo de seguridad crítico.

- Utilizaremos un archivo .env en la raíz del proyecto para almacenar datos sensibles como SMTP_HOST, SMTP_USER y SMTP_PASS.
- Si utilizas Gmail en producción, debes activar la Verificación en dos pasos y generar una "Contraseña de Aplicación" específica.

4. Pruebas en Desarrollo con Mailtrap

Para evitar el riesgo de enviar correos de prueba a clientes reales o que nuestras cuentas sean marcadas como *spam* durante la fase de desarrollo, utilizaremos Mailtrap.

Mailtrap es un servidor SMTP falso que captura todos tus correos salientes en una bandeja de entrada virtual.

Mientras estamos en desarrollo es aconsejable poder comprobar que todos los correos se envían y se reciben correctamente.

Crea una cuenta gratuita en [Mailtrap](#), obtener tus credenciales (Host, Puerto, Usuario y Password) y sustituirlas en tu archivo .env. Esto te permitirá previsualizar el diseño HTML del correo y verificar que los datos se renderizan correctamente antes de pasar a producción.

Busca los datos para realizar la configuración en PHPMailer. En la siguiente imagen se ven mis datos. No los copies, puesto que después de capturar esta imagen haré un reseteo y no te servirán.

17. Mis pedidos

Escribe el código necesario para que un cliente pueda consultar sus pedidos y el estado de cada uno de ellos.

18. Pagos(Paypal)

Investiga cómo añadir a tu aplicación el pago con Paypal.

En el siguiente archivo podrás ver cómo crear tu cuenta y hacer algunas pruebas con ella antes de crear el código de tu controlador.

[Descargar archivo pagos Paypal](#)

19. Confirmación Registro

Modifica el registro de usuarios para que sólo sea posible cuando el usuario haya confirmado su identidad mediante un correo con un token que recibirá.

JWT es una herramienta de estándar abierto cuyo objetivo es el intercambio de información entre dos sitios de forma segura y efectiva. Se suele usar para autenticar la identidad de un determinado usuario y comprobar no se ha modificado durante la transmisión. Además, se realiza con una serie de claims o privilegios. Esta información es verificada y se firma de forma digital.

Los privilegios están codificados en objetos de tipo JSON, que se incrustan dentro del payload o cuerpo de un mensaje que irá firmado digitalmente.

Es importante que antes de trabajar con JWT consultes su documentación:

- [Página oficial](#)
- [El estándar y su documentación y las librerías de PHP](#)

Para saber más: <https://openwebinars.net/blog/que-es-json-web-token-y-como-funciona/>

En el ejemplo que vamos a desarrollar usaremos la librería de `composer require firebase/php-jwt` . Puedes encontrar su documentación junto con ejemplos de uso [aquí](#).

Pasos a seguir